

Göstericiler (Pointer)

pointer = işaretçi = gösterici = gösterge

Değişken ve Bellek Adresi

Bilgisayarın ana belleği (RAM) sıralı kaydetme gözlerinden oluşmuştur. Her göze bir adres atanmıştır. Bu adreslerin değerleri 0 ile belleğin sahip olduğu üst değere bağlı olarak değişebilir. Örneğin 1 GB bir bellek, $1024 * 1024 * 1024 = 1073741824$ (bayt) adet gözden oluşur. Değişken tiplerinin bellekte işgal ettiği alanın bayt cinsinden uzunluğu `sizeof()` operatörüyle öğrenilir.

Bir programlama dilinde, belli bir tipte değişken tanımlanıp ve bir değer atandığında, o değişkene dört temel özellik eşlik eder:

1. değişkenin adı
2. değişkenin tipi
3. değişkenin sahip olduğu değer (içerik)
4. değişkenin bellekteki adresi

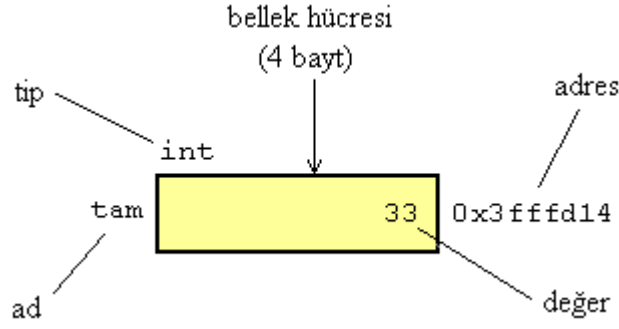
Örneğin `tam` adlı bir tamsayı değişkenini aşağıdaki gibi tanımladığımızı varsayalım:

```
int tam = 33;
```

Bu değişken için, `int` tipinde bellekte (genellikle herbiri 1 bayt olan 4 bayt büyüklüğünde) bir hücre ayrılır ve o hücreye 33 sayısı ikilik (binary) sayı sistemindeki karşılığı olan 4 baytlık (32 bitlik):

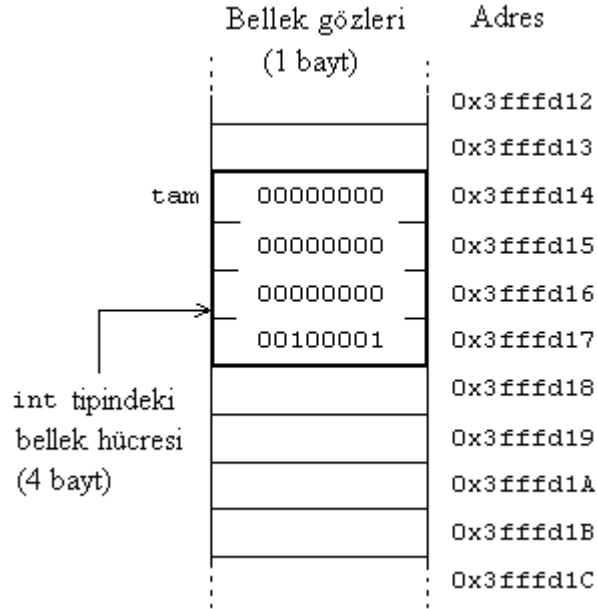
00000000 00000000 00000000 00100001

sayısı elektronik olarak yazılır. `tam` değişkenine ait dört temel özellik Şekil 1'deki gibi gösterilebilir:



Şekil 1: Bir değişkene eşlik eden dört temel özellik

Bellek adresleri genellikle onaltılık (hexadecimal) sayı sisteminde ifade edilir. 0x3fffd14 sayısı onluk (decimal) sayı sisteminde 67108116 sayına karşılık gelir. Bunun anlamı, `tam` değişkeni, program çalıştığı sürece, bellekte 67108116. - 67108120. numaralı gözler arasındaki 4 baytlık hücreyi işgal edecek olmasıdır. Şekil 1'deki gösterim, basit ama anlaşılır bir tasvirdir. Gerçekte, `int` tipindeki `tam` değişkeninin bellekteki yerleşimi ve içeriği (değeri) Şekil 2'de gösterildiği gibi olacaktır.



Şekil 2: `tam` adlı değişkenin bellekteki gerçek konumu ve ikilik düzendeki içeriği

Değişkenin saklı olduğu adres, `&` karakteri ile tanımlı *adres operatörü* ile öğrenilebilir. Bu operatör bir değişkenin önüne konursa, o değişkenin içeriği ile değil adresi ile ilgileniliyor anlamına gelir. Aşağıdaki program parçasının:

```
int tam = 33;

printf("icerik: %d\n", tam);
printf("adres : %p\n", &tam);
```

çıktısı:

```
icerik: 33
adres : 3fffd14
```

şeklindedir. Burada birinci satır `tam` değişkeninin içeriği, ikinci ise adresidir. Adres yazdırılırken `%p` tip belirleyicisinin kullanıldığına dikkat ediniz.

Gösterici Nedir?

Gösterici, bellek alanındaki bir gözün adresinin saklandığı değişkendir. Göstericilere veriler (yani değişkenlerin içeriği) değil de, o verilerin bellekte saklı olduğu hücrenin başlangıç adresleri atanır. Kısaca gösterici adres tutan bir değişkendir.

Bir gösterici, diğer değişkenler gibi, sayısal bir değişkendir. Bu sebeple kullanılmadan önce program içinde bildirilmelidir. Gösterici tipindeki değişkenler şöyle tanımlanır:

```
tip_adı *gösterici_adı;
```

Burada `tip_adı` herhangi bir C tip adı olabilir. Değişkenin önündeki `*` karakteri yönlendirme (indirection) operatörü olarak adlandırılır ve bu değişkenin veri değil bir adres bilgisi tutacağını işaret eder. Örneğin:

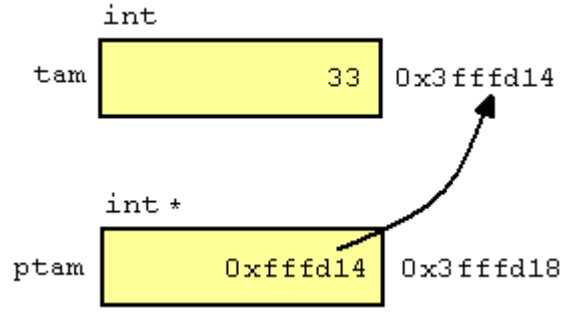
```
char *kr;           /* tek bir karakter için */
int *x;             /* bir tamsayı için */
float *deger, sonuc; /* deger gösterici tipinde, sonuc sıradan
bir gerçel değişkenler */
```

Yukarıda bildirilen göstericilerden; `kr` bir karakterin, `x` bir tamsayının ve `deger` bir gerçel sayının bellekte saklı olduğu yerlerin adreslerini tutar.

Bir göstericiye, bir değişkenin adresini atamak için adres operatörünü kullanabiliriz. Örneğin tamsayı tipindeki `tam` adlı bir değişken ve `ptam` bir gösterici olsun. Derleyicide, aşağıdaki gibi bir atama yapıldığında:

```
int *ptam, tam = 33;
.
.
.
ptam = &tam;
```

`ptam` göstericisinin `tam` değişkeninin saklandığı adresi tutacaktır. Bu durum Şekil 3'teki gibi tasvir edilir.



Şekil 3: Göstericinin bir değişkenin adresini göstermesi

Şekil 3'teki gösterimde, `ptam` göstericisinin içeriği `tam` değişkeninin içeriği (33) değil adresidir (0x3fffd14). Ayrıca, `ptam` değişkeni, bellekte başka bir hücrede saklandığına ve bu hücrenin `int` değil `int *` tipinde bir bölge olduğuna dikkat ediniz. Buraya kadar anlatılanlar, Program 1'de özetlenmiştir.

Program 1: Bir değişkenin içeriğini ve adresini ekrana yazdırma

```
01: /* 10prg01.c: ilk gösterici programı */
02:
03: #include <stdio.h>
04:
05: int main()
06: {
07:     int *ptam, tam = 33;
08:
09:     ptam = &tam;
10:
11:     printf("tam: icerik = %d\n", tam);
12:     printf("tam: adres = %p\n", &tam);
13:     printf("tam: adres = %p\n", ptam);
14:
15:     return 0;
16: }
```

7. satırda değişkenler bildirilmiştir. 9. satırdaki atama ile `tam` değişkeninin *adres*i, `ptam` göstericisine atanmıştır. Bu satırdan itibaren `ptam`, `tam` değişkeninin gösterir. 11. satırda `tam`'ın içeriği (33 sayısı), 12. ve 13. satırda `tam`'ın adresi, `%p` tip karakteri ile, ekrana yazdırılmıştır. Ekran çıktısı incelendiğinde, `&tam` ve `ptam` içereriğinin aynı anlamda olduğu görülür.

ÇIKTI

```
tam: icerik = 33
tam: adres = 0x3fffd14
tam: adres = 0x3fffd14
```

`tam` adlı değişkenin içeriğine `ptam` gösterici üzerinde de erişilebilir. Bunun için program içinde `ptam` değişkeninin önüne yönlendirme operatörü (*) koymak yeterlidir. Yani `*ptam`, `tam` değişkeninin adresini değil içeriğini tutar. Buna göre:

```
*ptam = 44;
```

komutuyla, `ptam`'ın adresini tuttuğu hücreye 44 değeri atanır. Bu durum, Program 2'de gösterilmiştir.

Program 2: *Bir değişkenin içeriğini ve adresini ekrana yazdırma*

```
01: /* 10prg02.c: ikinci gösterici programı */
02:
03: #include <stdio.h>
04:
05: int main()
06: {
07:     int *ptam, tam = 33;
08:
09:     ptam = &tam;    /* ptam -> tam */
10:
11:     printf("&tam = %p\n", &tam);
12:     printf("ptam = %p\n", ptam);
13:     printf("\n");
14:
15:     printf("tam = %d\n", tam);
16:     printf("*ptam = %d\n", *ptam);
17:     printf("\n");
18:
19:     *ptam = 44;    /* tam = 44 anlamında */
20:
21:     printf("tam = %d\n", tam);
22:     printf("*ptam = %d\n", *ptam);
23:
24:     return 0;
25: }
```

ÇIKTI

```
&tam = 0x3ffffd14
ptam = 0x3ffffd14
```

```
tam = 33
*ptam = 33
```

```
tam = 44
*ptam = 44
```

Özetle `ptam = &tam` atamasıyla:

- `*ptam` ve `tam`, `tam` adlı değişkenin içeriği ile ilgilidir.
- `ptam` ve `&tam`, `tam` adlı değişkenin adresi ile ilgilidir.
- `*` yönlendirme ve `&` adres operatörüdür.

Gösterici Aritmetiği

Göstericiler kullanılırken, bazen göstericinin gösterdiği adres taban alınıp, o adresten önceki veya sonraki adreslere erişilmesi istenebilir. Bu durum, göstericiler üzerinde, aritmetik işlemcilerin kullanılmasını gerektirir. Göstericiler üzerinde yalnızca toplama (+), çıkarma (-), bir arttırma (++) ve bir eksiltme (--) operatörleri işlemleri yapılabilir.

Aşağıdaki gibi üç tane gösterici bildirilmiş olsun:

```
char    *kar;  
int      *tam;  
double  *ger;
```

Bu göstericiler sırasıyla, bir karakter, bir tamsayı ve bir gerçel sayının bellekte saklanacağı adreslerini tutar. Herhangi bir anda, tuttukları adresler de sırasıyla 10000 (0x2710), 20000 (0x4e20) ve 30000 (0x7530) olsun. Buna göre aşağıdaki atama işlemlerinin sonucu:

```
kar++;  
tam++;  
ger++;
```

sırasıyla 10001 (0x2711), 20004 (0x4e24) ve 30008 (0x7538) olur. Bir göstericiye ekleme yapıldığında, o anda tuttuğu adres ile eklenen sayı doğrudan toplanmaz. Böyle olsaydı, bu atamaların sonuçları sırasıyla 10001, 20001 ve 30001 olurdu. Gerçekte, göstericiye bir eklemek, göstericinin gösterdiği yerdeki veriden hemen sonraki verinin adresini hesaplamaktır.

Genel olarak, bir göstericiye n sayısını eklemek (veya çıkarmak), bellekte gösterdiği veriden sonra (veya önce) gelen n . elemanın adresini hesaplamaktır. Buna göre aşağıdaki atamalar şöyle yorumlanır.

```
kar++;          /* kar = kar + sizeof(char) */  
tam = tam + 5;  /* tam = tam + 5*sizeof(int)  */  
ger = ger - 3;  /* ger = ger - 3*sizeof(double) */
```

Program 3, bu bölümde anlatılanları özetlemektedir.

Program 3: Gösterici aritmetiği

```
01: /* 10prg03.c: gösterici aritmetiği */  
02:  
03: #include <stdio.h>  
04:  
05:  
06: int main()  
07: {  
08:     char    *pk, k = 'a';  
09:     int      *pt, t = 22;  
10:     double  *pg, g = 5.5;  
11:  
12:     pk = &k;  
13:     pt = &t;  
14:     pg = &g;  
15:  
16:     printf("Önceki adresler: pk= %p pt= %p  
17: pg= %p \n", pk, pt, pg);  
18:  
19:     pk++;  
20:     pt--;  
21:     pg = pg + 10;  
22:
```

```
23:     printf("Sonraki adresler: pk= %p  pt= %p\n", pk, pt, pg);
24:     pg= %p \n", pk, pt, pg);
25:     return 0;
}
```

ÇIKTI

```
Önceki adresler: pk= 0xbfbbe88f  pt= 0xbfbbe888
pg= 0xbfbbe880
Sonraki adresler: pk= 0xbfbbe890  pt= 0xbfbbe884
pg= 0xbfbbe8d0
```

Gösterici ve Diziler Arasındaki İlişki

C dilinde göstericiler ve diziler arasında yakın bir ilişki vardır. Bir dizinin adı, dizinin ilk elemanının adresini saklayan bir göstericidir. Bu yüzden, bir dizinin herhangi bir elemanına gösterici ile de erişilebilir. Örneğin:

```
int kutle[5], *p, *q;
```

şeklinde bir bildirim yapılsın. Buna göre aşağıda yapılan atamalar geçerlidir:

```
p = &kutle[0];    /* birinci elemanın adresi p göstericisine
atandı */
p = kutle;        /* birinci elemanın adresi p göstericisine
atandı */
q = &kutle[4];    /* son elemanın adresi q göstericisine
atandı */
```

İlk iki satırdaki atamalar aynı anlamdadır. Dizi adı bir gösterici olduğu için, doğrudan aynı tipteki bir göstericiye atanabilir. Ayrıca, *i* bir tamsayı olmak üzere,

```
kutle[i];
```

ile

```
*(p+i);
```

aynı anlamdadır. Bunun sebebi, *p* göstericisi *kutle* dizisinin başlangıç adresini tutmuş olmasıdır. *p+i* işlemi ile *i+1*. elemanın adresi, ve **(p+i)* ile de bu adresteki değer hesaplanır.

NOT

Bir dizinin, *i*. elemanına erişmek için **(p+i)* işlemi yapılması zorunludur. Yani

```
*p+i; /* p nin gösterdiği değere (dizinin ilk
elemanına) i sayısını ekle */
```

```
*(p+i); /* p nin gösterdiği adresten i blok
ötedeki sayıyı hesapla */
```

anlamındadır. Çünkü, *** operatörü *+* operatörüne göre işlem önceliğine sahiptir.

Program 4'te tanımlanan fonksiyon kendine parameter olarak gelen n elemanlı bir dizinin aritmetik ortalamasını hesaplar.

Program 4: *Bir dizi ile gösterici arasındaki ilişki*

```
01: /* 10prg04.c: gösterici dizi ilişkisi */
02:
03: #include <stdio.h>
04:
05: double ortalama(double dizi[], int n);
06:
07: int main()
08: {
09:
10:     double a[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
11:     double o;
12:
13:     o = ortalama(a,5);
14:
15:     printf("Dizinin ortalaması = %lf\n",o);
16:
17:     return 0;
18: }
19:
20: double ortalama(double dizi[], int n)
21: {
22:     double *p, t=0.0;
23:     int i;
24:
25:     p = dizi;    /* veya p = &dizi[0] */
26:
27:     for(i=0; i<n; i++)
28:         t += *(p+i);
29:
30:     return (t/n);
31: }
```

ÇIKTI

Dizinin ortalaması = 3.300000

20. - 31. satırda tanımlanan fonksiyon aşağıdaki gibi de yazılabilirdi:

```
double ortalama(double dizi[], int n)
{
    double *p, t=0.0;

    for(p=dizi; p < &dizi[n]; p++)
        t += *p;

    return (t/n);
}
```

Bu fonksiyonda, döngü sayacı için (i değişkeni) kullanılmayıp, döngü içinde dizinin başlangıç adresi p göstericisine atanmış ve koşul kısmında adres karşılaştırılması yapılmıştır. Bu durumda döngü, p 'nin tuttuğu adresten başlar, ve p 'nin adresi dizinin son elemanının adresinden ($\&dizi[n-1]$) küçük veya eşit olduğu sürece çevrim yinelenir.

Fonksiyon Parametresi Olan Göstericiler

C (ve C++) programlama dilinde fonksiyon parametreleri değer geçerek (pass by value) yada adres geçerek (pass by reference) olarak geçilebilir. Bu şekilde geçirilen parametreler, fonksiyon içersinde değiştirilse bile, fonksiyon çağıldıktan sonra bu değişim çağrılan yerdeki değerini değiştirmez. Fakat, bir parametre adres geçerek aktarılırsa, fonksiyon içindeki değişiklikler geçilen parametreyi etkiler. Adres geçerek aktarım, gösterici kullanmayı zorunlu kılar.

Örneğin, Program 5'te fonksiyonlara değer ve adres geçerek aktarımın nasıl yapılacağı gösterilmiştir.

Program 5: *Bir değişkenin içeriğini ve adresini ekrana yazdırma*

```
01: /* Değer geçerek ve adres geçerek aktarım */
02:
03: #include <stdio.h>
04:
05: void f1(int ); /* iki fonksiyon */
06: void f2(int *);
07:
08: int main()
09: {
10:     int x = 55;
11:
12:     printf("x in degeri,\n");
13:     printf("Fonksiyonlar cagrilmadan once:
14: %d\n",x);
15:
16:     /* f1 fonksiyonu çağrılıyor...*/
17:     f1(x);
18:     printf("f1 cagirildikten sonra      :
19: %d\n",x);
20:
21:
22:     /* f2 fonksiyonu çağrılıyor...*/
23:     f2(&x);
24:     printf("f2 cagirildikten sonra      :
25: %d\n",x);
26:
27:     return 0;
28: }
29:
30:
31: /* Değer geçerek aktarım */
32: void f1(int n){
33:     n = 66;
34:     printf("f1 fonksiyonu icinde      :
35: %d\n",n);
36: }
37:
38: /* Adres geçerek aktarım */
39: void f2(int *n){
40:     *n = 77;
```

```
printf("f2 fonksiyonu icinde      :  
%d\n", *n);  
}
```

5. ve 6. satırlarda kendine geçilen parametrenin değerini alan `f1` fonksiyonu ve parametrenin adresini alan `f2` adlı iki fonksiyon örneği belirtilmiştir. 11. satırdaki `x` değişkeni 16. ve 21. satırlarda, `f1(x)` ve `f2(&x)` fonksiyonlarına, sırasıyla değer ve adres geçerek aktarılmıştır. `f1` içinde `x` (`n = 66;` işlemi ile) değişime uğramış, fakat çağırılma işleminin sonucunda, `x`'in değeri değişmemiştir. Ancak `f2` içinde `x`'in (`*n = 77` işlemi ile) değişimi, çağırıldıktan sonrada korunmuştur. Yani, adres geçerek yapılan aktarımda, `f2`'ye aktarılan değer değil adres olduğu için, yollanan `x` parametresi `f2` içinde değişikliğe uğrayacak ve bu değişim çağırıldığı 21. satırdan itibaren devam edecektir.

ÇIKTI

```
x in degeri,  
Fonksiyonlar cagrilmadan once: 55  
f1 fonksiyonu icinde      : 66  
f1 cagirildiktan sonra    : 55  
f2 fonksiyonu icinde      : 77  
f2 cagirildiktan sonra    : 77
```

NULL Gösterici

Bir göstericinin bellekte herhangi bir adresi göstermesi, veya önceden göstermiş olduğu adres iptal edilmesi istemirse `NULL` sabiti kullanılır. Bu sabit derleyicide ASCII karakter tablosunun ilk karakteridir ve `'\0'` ile sembolize edilir.

```
int *ptr, a = 12;  
.  
.  
ptr = &a;      /* ptr bellekte a değişkenin saklandığı yeri  
gösteriyor */  
.  
.  
ptr = NULL;    /* ptr bellekte hiç bir hücreyi göstermiyor */  
*ptr = 8       /* hata! NULL göstericinin gösterdiği yere bir  
değer atanamaz */
```